Algoritmos de optimización - Seminario

Nombre y Apellidos: **Pablo Carvajal Benitez - Javier Fernández Gordo**

Url: https://github.com/.../03MAIR---Algoritmos-de-Optimizacion---2019/tree/master/SEMINARIO
Problema:

> 1. Sesiones de doblaje
> **2. Organizar los horarios de partidos de La Liga**
> 2. Combinar cifras y operaciones

Problema 2: Organizar los horarios de partidos de La Liga

Desde La Liga de fútbol profesional se pretende organizar los horarios de los partidos de cada jornada.
Se conocen una serie de datos que deben permitir diseñar un algoritmo que realice la asignación de los partidos a los horarios disponibles de forma que **maximice la audiencia total**.

Horarios disponibles

Los horarios disponibles para disputar los partidos son los siguientes:

- **Viernes**: 20h
- **Sábado**: 12h, 16h, 18h, 20h
- **Domingo**: 12h, 16h, 18h, 20h
- **Lunes**: 20h

Clasificación de los equipos

Los equipos se clasifican en tres categorías según su número de seguidores, lo cual tiene una relación directa con la audiencia:

- **Categoría A**: 3 equipos
- **Categoría B**: 11 equipos
- **Categoría C**: 6 equipos

Cada partido queda definido por la categoría de los dos equipos que se enfrentan.

Audiencia base

Se conoce estadísticamente la audiencia (en millones) que genera cada partido **cuando se disputa el sábado a las 20h**, que es el mejor horario en todos los casos:

	Categoría A	Categoría B	Categoría C
Categoría A	2.0	1.3	1.0
Categoría B	0.9	0.75	0.47
Categoría C	0.47	0.47	0.47

Penalización por horario

Si el partido **no se disputa el sábado a las 20h**, la audiencia se reduce multiplicándola por el coeficiente correspondiente según el día y la hora:

Día / Hora	12h	16h	18h	20h
Viernes	-	-	-	0.4
Sábado	0.55	0.7	0.8	1.0
Domingo	0.45	0.75	0.85	1.0
Lunes	-	-	-	0.4

Coincidencia de horarios

Es posible que varios partidos coincidan en el mismo horario.
En ese caso, la audiencia de **cada partido** se verá afectada por una reducción porcentual en función del número de coincidencias:

Número de coincidencias	Reducción
0	0%
1	25%
2	45%

Número de coincidencias	Reducción
3	60%
4	70%
5	75%
6	78%
7	80%
8	80%

Restricciones obligatorias

- Debe asignarse **al menos un partido el viernes**
- Debe asignarse **al menos un partido el lunes**

Objetivo del problema

Diseñar un algoritmo que asigne los partidos a los horarios disponibles de forma que se **maximice la audiencia total de la jornada**, teniendo en cuenta:

- La audiencia base del partido según las categorías de los equipos
- La penalización por el horario asignado
- La penalización por coincidencia de horarios
- Las restricciones obligatorias del viernes y lunes

Resumen de características solicitadas

Franjas horarias disponibles (10 en total)

#	Franja	Coeficiente
0	Viernes 20h	0.40
1	Sábado 12h	0.55
2	Sábado 16h	0.70
3	Sábado 18h	0.80
4	Sábado 20h	1.00 ← referencia
5	Domingo 12h	0.45
6	Domingo 16h	0.75
7	Domingo 18h	0.85
8	Domingo 20h	1.00
9	Lunes 20h	0.40

Clasificación de equipos

- **Categoría A:** 3 equipos
- **Categoría B:** 11 equipos
- **Categoría C:** 6 equipos

Audiencia base (Sábado 20h, en millones)

	A	B	C
A	2.0	1.3	1.0
B	—	0.75	0.47
C	—	—	0.47

Penalización por coincidencias

Coincidencias	Reducción	Coeficiente
0	0%	1.00
1	25%	0.75
2	45%	0.55
3	60%	0.40
4	70%	0.30
5	75%	0.25
6	78%	0.22
7	80%	0.20
8	80%	0.20

Restricciones obligatorias

- Al menos **1 partido el viernes**
- Al menos **1 partido el lunes**

Objetivo

Maximizar la audiencia total de la jornada:

$$FO = \sum_i A_i \times c_{horario} \times c_{coincidencia}$$

Problema 2. Organizar los horarios de partidos de La Liga

Haz doble clic (o pulsa Intro) para editar

(*)¿Cuantas posibilidades hay sin tener en cuenta las restricciones?

¿Cuantas posibilidades hay teniendo en cuenta todas las restricciones.

Espacio de soluciones

Cada solución asigna cada uno de los 10 partidos a una de las 10 franjas horarias disponibles.

Sin restricciones

Como cualquier partido puede ir a cualquier franja independientemente del resto, tenemos:

$$\text{Posibilidades} = 10^{10} = 10.000.000.000$$

Son 10.000 millones de combinaciones. Lo cúal es imposible explorarlas todas con fuerza bruta.

Con restricciones

Las restricciones del enunciado obligan a tener:

- Al menos 1 partido en Viernes 20h (franja 0)
- Al menos 1 partido en Lunes 20h (franja 9)

Usando el principio de inclusión-exclusión:

$$\text{Válidas} = 10^{10} - 9^{10} - 9^{10} + 8^{10}$$

$$= 10.000.000.000 - 3.486.784.401 - 3.486.784.401 + 1.073.741.824 \approx 4.100.173.022$$

Aun así son unos 4.000 millones de soluciones válidas, lo que sigue siendo imposible para fuerza bruta.

```
Empieza a programar o a crear código con IA.
```

Modelo para el espacio de soluciones
(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, arguentalo)

Estructura de datos

Para abordar el problema, hemos evaluado distintas estructuras de datos. Aunque inicialmente planteamos el uso de diccionarios para mapear partidos y horarios, pero llegamos a la conclusión de que esta opción dificultaba el seguimiento de los partidos que coinciden en la misma franja y reducía el rendimiento del algoritmo.

Finalmente, hemos optado por una representación basada en una lista de enteros de longitud fija (N=10). Con esta solución, el índice de la lista nos permite identificar cada partido de forma única, mientras que el valor almacenado (v∈[0,9]) codifica el horario asignado. Esta estructura nos permite simplificar los operadores de vecindad y agilizar el cómputo de la función objetivo.

```
asig = [h_0, h_1, ..., h_9]   # 10 enteros, cada uno entre 0 y 9
```

Donde `asig[i]` indica la franja horaria asignada al partido `i`.

Al principio consideramos usar un diccionario `{partido: franja}`, pero en cuanto comenzamos a codificar la función objetivo nos dimos cuenta de que una lista simple es más sencilla de utilizar: el índice ya actúa como identificador del partido, el acceso es O(1) y resulta trivial generar vecinos cambiando un único elemento.

Los datos del problema los estructuramos así:

```
franjas = [
    ("Viernes 20h",  0.40),  # 0
    ("Sabado 12h",   0.55),  # 1
    ...
    ("Sabado 20h",   1.00),  # 4  <- referencia
    ...
    ("Lunes 20h",    0.40),  # 9
]

aud_base = {
    ("A","A"): 2.0,
    ("A","B"): 1.3, ("B","A"): 1.3,
    ...
}

pen_coinc = {0: 1.00, 1: 0.75, 2: 0.55, 3: 0.40,
             4: 0.30, 5: 0.25, 6: 0.22, 7: 0.20, 8: 0.20}
```

Hemos definido la jornada de ejemplo como una estructura de lista que contiene los 10 enfrentamientos. Cada elemento es una tupla con las categorías de los dos equipos (A, B o C), lo que nos permite mapear fácilmente los datos con el diccionario de audiencias:

```
jornada = [("A","B"), ("A","C"), ("B","B"), ("B","C"), ...]
```

Según el modelo para el espacio de soluciones
(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Función objetivo

Dado que el objetivo es optimizar los ingresos por visualizaciones, nos enfrentamos a un problema de **maximización**.

Para evaluar la calidad de cada solución propuesta por nuestro algoritmo, hemos diseñado una función objetivo que cuantifica la audiencia total de la jornada (en millones de espectadores). Esta función suma la audiencia de cada partido tras aplicar los factores de corrección por horario y por solapamiento:

Para cada partido `i` con franja asignada `h_i`, su audiencia real es:

$$\text{audiencia}_i = A_i \times c_{h_i} \times p_{n_{h_i}}$$

donde:

- A_i es la audiencia base según las categorías de los equipos (en millones)
- c_{h_i} es el coeficiente del horario asignado (va de 0.40 a 1.00)
- $p_{n_{h_i}}$ es la penalización por coincidir con otros partidos en la misma franja

La función objetivo suma las audiencias de los 10 partidos:

$$FO(\mathbf{h}) = \sum_{i=0}^{9} A_i \times c_{h_i} \times p_{n_{h_i}}$$

Para que una solución sea válida debe cumplir:

- Al menos un partido en la franja 0 (Viernes 20h)
- Al menos un partido en la franja 9 (Lunes 20h)

El objetivo es encontrar la asignación $\mathbf{h}^*$ que maximice FO entre todas las soluciones válidas.

```
Empieza a programar o a crear código con IA.
```

Diseña un algoritmo para resolver el problema por fuerza bruta

```
from itertools import product
from collections import Counter

# 1. Parámetros globales y estructuras auxiliares
# Definimos las franjas horarias con sus respectivos coeficientes de ponderación
franjas = [
    ("Viernes 20h",  0.40),  # 0 <- obligatorio
    ("Sabado 12h",   0.55),  # 1
    ("Sabado 16h",   0.70),  # 2
```

```
    ("Sabado 18h",   0.80),  # 3
    ("Sabado 20h",   1.00),  # 4 <- referencia (mejor horario)
    ("Domingo 12h",  0.45),  # 5
    ("Domingo 16h",  0.75),  # 6
    ("Domingo 18h",  0.85),  # 7
    ("Domingo 20h",  1.00),  # 8
    ("Lunes 20h",    0.40),  # 9 <- obligatorio
]

# Audiencias base por tipo de enfrentamiento.
# Hemos incluido ambas permutaciones (A,B y B,A) para agilizar la consulta
# y evitar comprobaciones adicionales dentro del bucle crítico de la función objetivo.
aud_base = {
    ("A","A"): 2.0,
    ("A","B"): 1.3, ("B","A"): 1.3,
    ("A","C"): 1.0, ("C","A"): 1.0,
    ("B","B"): 0.75,
    ("B","C"): 0.47, ("C","B"): 0.47,
    ("C","C"): 0.47,
}

# Coeficiente de reducción según cuántos partidos coinciden en la misma franja.
# Usamos un diccionario para aplicar directamente el porcentaje de reducción
# sobre la audiencia base según el número de coincidencias.
pen_coinc = {0: 1.00, 1: 0.75, 2: 0.55, 3: 0.40,
             4: 0.30, 5: 0.25, 6: 0.22, 7: 0.20, 8: 0.20}

def penalizacion(n):
    # Calcula la reducción de audiencia restando 1 al total de partidos
    # en la franja para obtener el número de coincidencias reales.
    return pen_coinc.get(n - 1, 0.20)

def fo(asig, partidos):
    # Función objetivo que calcula la audiencia total de una jornada.
    # Usamos Counter para saber cuántos partidos hay por horario y aplicar las penalizaciones.
    cnt = Counter(asig)
    total = 0.0
    for i, h in enumerate(asig):
        total += aud_base[partidos[i]] * franjas[h][1] * penalizacion(cnt[h])
    return round(total, 4)

def valida(asig):
    # Comprobamos que la solución cumple las restricciones del enunciado:
    # debe haber al menos un partido el viernes (0) y otro el lunes (9).
    return 0 in asig and 9 in asig

def fuerza_bruta(partidos, n_franjas=10):
    # Algoritmo de fuerza bruta que explora todas las combinaciones posibles.
    best, best_fo, n_eval = None, -1, 0
    for asig in product(range(n_franjas), repeat=len(partidos)):
        if valida(asig):
            v = fo(list(asig), partidos)
            n_eval += 1
            if v > best_fo:
                best_fo, best = v, list(asig)
    return best, best_fo, n_eval

# 2. Ejecución y Pruebas
# Pruebas con una sub-instancia pequeña
# Hemos reducido la jornada a 4 partidos para que la fuerza bruta sea viable
demo = [("A","B"), ("B","B"), ("B","C"), ("C","C")]

sol, res, n_eval = fuerza_bruta(demo)

print("Combinaciones válidas evaluadas:", n_eval)
print("El máximo de audiencia conseguido es:", res, "millones")
print("La mejor asignación de horarios es:", sol)

print("\nDetalle de la jornada:")
for i in range(len(sol)):
    horario_idx = sol[i]
    nombre_horario = franjas[horario_idx][0]
    partido = demo[i]
    print("Partido", i, partido, "se juega el:", nombre_horario)
```

```
Combinaciones válidas evaluadas: 974
El máximo de audiencia conseguido es: 2.426 millones
La mejor asignación de horarios es: [4, 8, 0, 9]

Detalle de la jornada:
Partido 0 ('A', 'B') se juega el: Sabado 20h
Partido 1 ('B', 'B') se juega el: Domingo 20h
Partido 2 ('B', 'C') se juega el: Viernes 20h
```

```
Partido 3 ('C', 'C') se juega el: Lunes 20h
```

```
Empieza a programar o a crear código con IA.
```

Calcula la complejidad del algoritmo por fuerza bruta

Complejidad del algoritmo de fuerza bruta

Con $N = 10$ partidos y $H = 10$ franjas, el total de combinaciones posibles es:

$$H^N = 10^{10} = 10.000.000.000$$

Para cada una hay que verificar las restricciones y calcular la función objetivo, ambas operaciones en $O(N)$. La complejidad total es por tanto:

$$O(H^N \cdot N)$$

Con $H = N = 10$ eso son del orden de 10^{11} operaciones elementales.

Si consideramos que en un entorno de ejecución estándar con Python podemos procesar aproximadamente 106 evaluaciones por segundo, el tiempo estimado para explorar todo el espacio sería de unas 2.7 horas.

Por este motivo, en la prueba anterior limitamos la jornada a 4 partidos (104 combinaciones), para poder verificar que el código es correcto sin esperar horas.

Esto confirma que la fuerza bruta no es una opción práctica para el problema completo y que hay que recurrir a algún método heurístico.

(*)Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Por qué la Búsqueda Tabú mejora la fuerza bruta

La fuerza bruta explora todas las combinaciones posibles del espacio de soluciones, lo que con 10 partidos y 10 franjas supone 10^{10} estados. Es computacionalmente inviable para la instancia completa del problema.

La Búsqueda Tabú en cambio no explora todo el espacio: parte de una solución aleatoria válida y en cada iteración solo evalúa los 90 vecinos inmediatos (cambiar la franja de un partido). Elige el mejor de esos vecinos y se mueve a él, aunque no mejore la solución actual. La clave es la **lista tabú**: un registro de los últimos movimientos realizados que están temporalmente prohibidos. Esto evita que el algoritmo dé vueltas en círculo y le obliga a explorar zonas nuevas del espacio que de otro modo no visitaría.

Además, incorporamos el **criterio de aspiración**: si un movimiento tabú conduciría a la mejor solución encontrada hasta ahora, se acepta igualmente. Esto garantiza que nunca descartamos una mejora real por culpa de la restricción tabú.

En resumen, en lugar de explorar los 10^{10} estados, exploramos como máximo $500 \times 90 = 45.000$, lo que supone una reducción drástica manteniendo una calidad de solución razonablemente buena.

```
import random
from collections import deque

# Búsqueda Tabú
# La idea es parecida a la búsqueda local clásica, pero con una diferencia importante:
# llevamos un registro de los últimos movimientos realizados (lista tabú) para no repetirlos.
# Esto obliga al algoritmo a explorar zonas nuevas del espacio aunque no haya mejora inmediata,
# lo que le permite escapar de los óptimos locales en los que se quedaría atascado una búsqueda sin memoria.

def asig_aleatoria(n=10, h=10):
    # Generamos una asignación al azar y validamos que cumpla las restricciones de la jornada
    while True:
        asig = [random.randint(0, h - 1) for _ in range(n)]
        if valida(asig):
            return asig

def vecinos(asig, h=10):
    # Definimos el vecindario mediante el cambio de la franja horaria de un único partido.
    # Junto a cada vecino guardamos el movimiento que lo generó: (partido, franja_nueva)
    # para poder añadirlo a la lista tabú si lo elegimos.
    res = []
    for i in range(len(asig)):
        for franja in range(h):
            if franja != asig[i]:
                v = asig[:]
                v[i] = franja
                res.append((v, (i, franja)))
```

```
    return res

def busqueda_tabu(partidos, h=10, max_iter=500, tamanio_tabu=15):
    # Iniciamos desde una solución aleatoria válida
    asig = asig_aleatoria(len(partidos), h)
    actual_fo = fo(asig, partidos)

    # Guardamos siempre la mejor solución encontrada, aunque no sea la actual
    mejor_asig = asig[:]
    mejor_fo   = actual_fo

    # La lista tabú almacena los últimos movimientos prohibidos.
    # Usamos deque con tamaño fijo para que los movimientos más antiguos
    # se expulsen solos cuando la lista se llena.
    lista_tabu = deque(maxlen=tamanio_tabu)

    for _ in range(max_iter):
        candidatos = vecinos(asig, h)

        # Elegimos el mejor vecino válido que no esté en la lista tabú.
        # Si ninguno cumple, nos quedamos con el mejor aunque esté prohibido
        # (criterio de aspiración: si mejora el mejor global, se acepta igualmente).
        mejor_vecino     = None
        mejor_fo_vecino  = -1
        mov_elegido      = None

        for v, mov in candidatos:
            if not valida(v):
                continue
            v_fo = fo(v, partidos)
            # Aceptamos el movimiento si no está en tabú O si mejora el mejor global (aspiración)
            if mov not in lista_tabu or v_fo > mejor_fo:
                if v_fo > mejor_fo_vecino:
                    mejor_vecino    = v
                    mejor_fo_vecino = v_fo
                    mov_elegido     = mov

        # Si no encontramos ningún candidato viable, terminamos
        if mejor_vecino is None:
            break

        # Nos movemos al mejor vecino aunque sea peor que el actual
        asig       = mejor_vecino
        actual_fo  = mejor_fo_vecino
        lista_tabu.append(mov_elegido)

        # Actualizamos el mejor global si hemos mejorado
        if actual_fo > mejor_fo:
            mejor_fo   = actual_fo
            mejor_asig = asig[:]

    return mejor_asig, mejor_fo

# Ejecución final del algoritmo con los parámetros configurados para la jornada completa
print("Ejecutando Búsqueda Tabú...")
jornada_completa = [("A","B"), ("A","C"), ("B","B"), ("B","C"), ("B","C"),
                    ("B","B"), ("C","C"), ("B","C"), ("A","B"), ("A","A")]

sol, aud = busqueda_tabu(jornada_completa)

print("Resultado Búsqueda Tabú:")
print(f"Audiencia total: {aud} millones")
print("Asignación final:")

for i in range(len(sol)):
    partido = jornada_completa[i]
    horario = franjas[sol[i]][0]
    print(f"Partido {i+1} ({partido[0]}-{partido[1]}): {horario}")
```

```
Ejecutando Búsqueda Tabú...
Resultado Búsqueda Tabú:
Audiencia total: 6.9995 millones
Asignación final:
Partido 1 (A-B): Domingo 16h
Partido 2 (A-C): Sabado 18h
Partido 3 (B-B): Domingo 18h
Partido 4 (B-C): Lunes 20h
Partido 5 (B-C): Sabado 12h
Partido 6 (B-B): Viernes 20h
Partido 7 (C-C): Sabado 16h
Partido 8 (B-C): Domingo 12h
Partido 9 (A-B): Domingo 20h
```

```
Partido 10 (A-A): Sabado 20h
```

```
Empieza a programar o a crear código con IA.
```

(*)Calcula la complejidad del algoritmo

Complejidad del algoritmo mejorado

Para mejorar la fuerza bruta se utiliza una **Búsqueda Tabú**, que es una técnica de búsqueda local con memoria. La idea principal es explorar el espacio de soluciones evitando volver a estados recientes y así no quedarse atrapado en óptimos locales.

En cada iteración se evalúa todo el vecindario de la solución actual. En este problema, el vecindario tiene $N \times (H-1) = 10 \times 9 = 90$ vecinos, ya que se cambia la franja horaria de un único partido cada vez. De todos ellos se selecciona el mejor movimiento que no esté prohibido por la lista tabú, aunque no mejore la solución actual.

Si el algoritmo se ejecuta durante un máximo de I iteraciones, su complejidad es:

$$O(I \times N \times H)$$

Con los valores usados ($I = 500$, $N = 10$, $H = 10$), se realizan como máximo 50.000 evaluaciones de la función objetivo, muy lejos de las 10^{10} combinaciones de la fuerza bruta.

La lista tabú evita ciclos y permite diversificar la búsqueda sin necesidad de reinicios aleatorios. En este caso, un tamaño de lista tabú de 15 movimientos da buenos resultados. Aunque no garantiza el óptimo global, la calidad de las soluciones obtenidas es buena con un coste computacional asumible.

```
Empieza a programar o a crear código con IA.
```

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

```
import random
from collections import Counter

# Definimos las franjas horarias con sus respectivos pesos de audiencia
franjas = [
    ("Viernes 20h",  0.40),
    ("Sabado 12h",   0.55),
    ("Sabado 16h",   0.70),
    ("Sabado 18h",   0.80),
    ("Sabado 20h",   1.00),
    ("Domingo 12h",  0.45),
    ("Domingo 16h",  0.75),
    ("Domingo 18h",  0.85),
    ("Domingo 20h",  1.00),
    ("Lunes 20h",    0.40),
]

# Estructura para consultar la audiencia base según el cruce de categorías
aud_base = {
    ("A","A"): 2.0,
    ("A","B"): 1.3, ("B","A"): 1.3,
    ("A","C"): 1.0, ("C","A"): 1.0,
    ("B","B"): 0.75,
    ("B","C"): 0.47, ("C","B"): 0.47,
    ("C","C"): 0.47,
}

# Coeficientes de penalización por solapamiento de partidos en la misma franja
pen_coinc = {0: 1.00, 1: 0.75, 2: 0.55, 3: 0.40,
             4: 0.30, 5: 0.25, 6: 0.22, 7: 0.20, 8: 0.20}

def penalizacion(n):
    # Restamos 1 al total de partidos para saber cuántos coinciden realmente y aplicamos la reducción.
    return pen_coinc.get(n - 1, 0.20)

def fo(asig, partidos):
    # Calculamos la audiencia total de la jornada aplicando todos los coeficientes
    cnt = Counter(asig)
    total = 0.0
    for i, h in enumerate(asig):
        total += aud_base[partidos[i][2]] * franjas[h][1] * penalizacion(cnt[h])
    return round(total, 4)

def valida(asig):
    # Comprobamos que se cumpla la restricción obligatoria de partidos en Viernes y Lunes
    return 0 in asig and 9 in asig
```

```
# Listado de equipos de LaLiga con su clasificación por categoría de audiencia
equipos = [
    ("Real Madrid", "A"), ("Barcelona", "A"), ("Atlético", "A"),
    ("Sevilla", "B"), ("Alavés", "B"), ("Athletic", "B"),
    ("Villarreal", "B"), ("Real Sociedad", "B"), ("Valencia", "B"),
    ("Getafe", "B"), ("Espanyol", "B"), ("Betis", "B"),
    ("Celta", "B"), ("Levante", "B"),
    ("Leganés", "C"), ("Valladolid", "C"), ("Eibar", "C"),
    ("Granada", "C"), ("Mallorca", "C"), ("Osasuna", "C")
]

# Mostramos la distribución inicial de equipos
print("Equipos por categoria:")
for cat in ["A", "B", "C"]:
    grupo = [e[0] for e in equipos if e[1] == cat]
    print(f"Cat {cat}: {', '.join(grupo)}")

def generar_jornada(equipos, seed=None):
    # Mezclamos los equipos para generar una jornada aleatoria
    if seed is not None:
        random.seed(seed)
    lista = equipos[:]
    random.shuffle(lista)
    jornada = []
    # Emparejamos los equipos de dos en dos para formar los 10 partidos
    for i in range(0, 20, 2):
        loc, cat_l = lista[i]
        vis, cat_v = lista[i + 1]
        tipo = tuple(sorted([cat_l, cat_v]))
        jornada.append((loc, vis, tipo))
    return jornada

# Generamos la instancia de la jornada que utilizaremos en las pruebas
jornada = generar_jornada(equipos, seed=42)

print("\nPartidos de la jornada generada:")
for i, (loc, vis, tipo) in enumerate(jornada):
    base = aud_base[tipo]
    print(f"{i+1}. {loc} vs {vis} ({tipo[0]}-{tipo[1]}) -> {base}M")

# Calculamos la audiencia base teórica sin penalizaciones
total_base = sum(aud_base[p[2]] for p in jornada)
print(f"\nAudiencia base total (sin solapamientos): {total_base:.2f}M")

# Resumen de los tipos de enfrentamientos
tipos = Counter(p[2] for p in jornada)
print("Frecuencia de enfrentamientos:", dict(tipos))
```

```
Equipos por categoria:
Cat A: Real Madrid, Barcelona, Atlético
Cat B: Sevilla, Alavés, Athletic, Villarreal, Real Sociedad, Valencia, Getafe, Espanyol, Betis, Celta, Levante
Cat C: Leganés, Valladolid, Eibar, Granada, Mallorca, Osasuna

Partidos de la jornada generada:
1. Osasuna vs Athletic (B-C) -> 0.47M
2. Leganés vs Alavés (B-C) -> 0.47M
3. Getafe vs Levante (B-B) -> 0.75M
4. Valladolid vs Mallorca (C-C) -> 0.47M
5. Villarreal vs Celta (B-B) -> 0.75M
6. Granada vs Espanyol (B-C) -> 0.47M
7. Barcelona vs Betis (A-B) -> 1.3M
8. Atlético vs Eibar (A-C) -> 1.0M
9. Real Sociedad vs Valencia (B-B) -> 0.75M
10. Real Madrid vs Sevilla (A-B) -> 1.3M

Audiencia base total (sin solapamientos): 7.73M
Frecuencia de enfrentamientos: {('B', 'C'): 3, ('B', 'B'): 3, ('C', 'C'): 1, ('A', 'B'): 2, ('A', 'C'): 1}
```

Empieza a programar o a crear código con IA.

Aplica el algoritmo al juego de datos generado

```
import time

# Extraemos solo las categorías de la jornada para facilitar el proceso de optimización
cats = [p[2] for p in jornada]

def asig_aleatoria(n=10, h=10):
    # Generamos soluciones aleatorias hasta que una cumpla las restricciones de la liga
    while True:
        asig = [random.randint(0, h - 1) for _ in range(n)]
```

```
        if valida(asig):
            return asig

def vecinos(asig, h=10):
    # Construimos el vecindario modificando la franja de un partido cada vez.
    # Devolvemos también el movimiento realizado para poder registrarlo en la lista tabú.
    res = []
    for i in range(len(asig)):
        for franja in range(h):
            if franja != asig[i]:
                v = asig[:]
                v[i] = franja
                res.append((v, (i, franja)))
    return res

def busqueda_tabu(partidos, h=10, max_iter=500, tamanio_tabu=15):
    # Iniciamos desde un punto aleatorio válido
    asig = asig_aleatoria(len(partidos), h)
    actual_fo = fo(asig, [(None, None, c) for c in partidos])

    mejor_asig = asig[:]
    mejor_fo   = actual_fo

    # La lista tabú bloquea los últimos movimientos para forzar la exploración de zonas nuevas
    lista_tabu = deque(maxlen=tamanio_tabu)

    for _ in range(max_iter):
        candidatos = vecinos(asig, h)

        mejor_vecino    = None
        mejor_fo_vecino = -1
        mov_elegido     = None

        for v, mov in candidatos:
            if not valida(v):
                continue
            v_fo = fo(v, [(None, None, c) for c in partidos])
            # Aceptamos si no está en tabú, o si mejora el mejor global (criterio de aspiración)
            if mov not in lista_tabu or v_fo > mejor_fo:
                if v_fo > mejor_fo_vecino:
                    mejor_vecino    = v
                    mejor_fo_vecino = v_fo
                    mov_elegido     = mov

        if mejor_vecino is None:
            break

        # Nos movemos al mejor candidato aunque empeore respecto al estado actual
        asig      = mejor_vecino
        actual_fo = mejor_fo_vecino
        lista_tabu.append(mov_elegido)

        # Guardamos la mejor solución global encontrada hasta ahora
        if actual_fo > mejor_fo:
            mejor_fo   = actual_fo
            mejor_asig = asig[:]

    return mejor_asig, mejor_fo

# --- Comparación sobre la sub-instancia demo (4 partidos) ---
# Usamos la misma jornada reducida que en la fuerza bruta para poder comparar
# ambos algoritmos sobre exactamente el mismo problema.
# Adaptamos demo al formato (loc, vis, tipo) que usa fo en esta celda,
# ya que la fuerza bruta de cell-14 usaba tuplas de 2 elementos y aquí usamos tuplas de 3.
demo_ext = [(None, None, p) for p in demo]
demo_cats = [p[2] for p in demo_ext]

print("Comparación sobre la jornada demo (4 partidos):")

sol_fb, fo_fb, _ = fuerza_bruta(demo_ext)
print(f"  Fuerza bruta:     {fo_fb:.4f} M  (óptimo garantizado)")

random.seed(42)
sol_bt, fo_bt_demo = busqueda_tabu(demo_cats, max_iter=500, tamanio_tabu=15)
print(f"  Búsqueda Tabú:    {fo_bt_demo:.4f} M")
print(f"  Diferencia:       {abs(fo_bt_demo - fo_fb):.4f} M ({abs((fo_bt_demo/fo_fb)-1)*100:.2f}% del óptimo)")

# --- Aplicación a la jornada completa (10 partidos) ---
# Para la jornada completa la fuerza bruta no es viable, así que ejecutamos
# solo la Búsqueda Tabú y mostramos el resultado.
print("\nAplicación a la jornada completa (10 partidos):")
print("Configuración de la Jornada:")
```

```
for i, (loc, vis, tipo) in enumerate(jornada):
    print(f" {i+1}. {loc} vs {vis} [{tipo[0]}-{tipo[1]}]")

t0 = time.time()
asig_bt, fo_bt = busqueda_tabu(cats, max_iter=500, tamanio_tabu=15)
t1 = time.time()

print(f"\nResultado Búsqueda Tabú: {fo_bt:.4f} M (Tiempo: {t1-t0:.2f}s)")

print("\nAsignación Detallada:")
cnt = Counter(asig_bt)
for i, h in enumerate(asig_bt):
    loc, vis, tipo = jornada[i]
    # Desglosamos el cálculo para verificar los coeficientes aplicados
    base = aud_base[tipo]
    ch = franjas[h][1]
    cc = penalizacion(cnt[h])
    aud = base * ch * cc
    print(f" {i+1}. {loc}-{vis}: {franjas[h][0]} | Base: {base} * Ch: {ch} * Cc: {cc} = {aud:.4f}M")

print(f"\nAUDIENCIA TOTAL FINAL: {fo_bt:.4f} M")

print("\nOcupación de franjas:")
for h_idx, count in sorted(cnt.items()):
    print(f" - {franjas[h_idx][0]}: {count} partido(s)")
```

```
Comparación sobre la jornada demo (4 partidos):
  Fuerza bruta:     2.4260 M  (óptimo garantizado)
  Búsqueda Tabú:    2.2580 M
  Diferencia:       0.1680 M (6.92% del óptimo)

Aplicación a la jornada completa (10 partidos):
Configuración de la Jornada:
 1. Osasuna vs Athletic [B-C]
 2. Leganés vs Alavés [B-C]
 3. Getafe vs Levante [B-B]
 4. Valladolid vs Mallorca [C-C]
 5. Villarreal vs Celta [B-B]
 6. Granada vs Espanyol [B-C]
 7. Barcelona vs Betis [A-B]
 8. Atlético vs Eibar [A-C]
 9. Real Sociedad vs Valencia [B-B]
 10. Real Madrid vs Sevilla [A-B]

Resultado Búsqueda Tabú: 5.9385 M (Tiempo: 0.26s)

Asignación Detallada:
 1. Osasuna-Athletic: Viernes 20h | Base: 0.47 * Ch: 0.4 * Cc: 1.0 = 0.1880M
 2. Leganés-Alavés: Sabado 12h | Base: 0.47 * Ch: 0.55 * Cc: 1.0 = 0.2585M
 3. Getafe-Levante: Domingo 16h | Base: 0.75 * Ch: 0.75 * Cc: 1.0 = 0.5625M
 4. Valladolid-Mallorca: Domingo 12h | Base: 0.47 * Ch: 0.45 * Cc: 1.0 = 0.2115M
 5. Villarreal-Celta: Sabado 16h | Base: 0.75 * Ch: 0.7 * Cc: 1.0 = 0.5250M
 6. Granada-Espanyol: Lunes 20h | Base: 0.47 * Ch: 0.4 * Cc: 1.0 = 0.1880M
 7. Barcelona-Betis: Sabado 20h | Base: 1.3 * Ch: 1.0 * Cc: 1.0 = 1.3000M
 8. Atlético-Eibar: Domingo 20h | Base: 1.0 * Ch: 1.0 * Cc: 1.0 = 1.0000M
 9. Real Sociedad-Valencia: Sabado 18h | Base: 0.75 * Ch: 0.8 * Cc: 1.0 = 0.6000M
 10. Real Madrid-Sevilla: Domingo 18h | Base: 1.3 * Ch: 0.85 * Cc: 1.0 = 1.1050M

AUDIENCIA TOTAL FINAL: 5.9385 M

Ocupación de franjas:
 - Viernes 20h: 1 partido(s)
 - Sabado 12h: 1 partido(s)
 - Sabado 16h: 1 partido(s)
 - Sabado 18h: 1 partido(s)
 - Sabado 20h: 1 partido(s)
 - Domingo 12h: 1 partido(s)
 - Domingo 16h: 1 partido(s)
 - Domingo 18h: 1 partido(s)
 - Domingo 20h: 1 partido(s)
 - Lunes 20h: 1 partido(s)
```

Empieza a programar o a crear código con IA.

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

Referencias

1. **Stuart Russell y Peter Norvig** — *Artificial Intelligence: A Modern Approach* (4ª ed., 2020). El capítulo 4 sobre búsqueda local fue la referencia principal para entender los fundamentos de los métodos de búsqueda local, y el capítulo 4.5 explica la Búsqueda Tabú en el contexto de estos métodos.

2. **Fred Glover** — *Tabu Search—Part I*, ORSA Journal on Computing, 1989. El artículo original donde se introduce la Búsqueda Tabú y se explica el criterio de aspiración que hemos implementado.
3. **Apuntes del curso** — *Algoritmos de Optimización*, O3MAIR.
4. **Python Documentation** — `itertools.product`, `collections.Counter`, `collections.deque`, `random`. https://docs.python.org/3/library/
5. **Wikipedia** — Artículos sobre LaLiga y el sistema de competición para entender la distribución real de equipos por categorías.

Describe brevemente las lineas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Posibles mejoras y líneas de trabajo futuro

Mejoras al algoritmo

La principal limitación de la Búsqueda Tabú tal como la hemos implementado es que depende bastante del punto de partida inicial y del tamaño de la lista tabú. Hay varias formas de mejorarlo:

- **Multi-start Tabú**: lanzar varias ejecuciones desde puntos de partida aleatorios distintos y quedarse con la mejor solución. Combina la diversificación del multi-start con la capacidad de escapar de óptimos locales de Tabú.
- **Intensificación y diversificación adaptativa**: ajustar dinámicamente el tamaño de la lista tabú según el progreso del algoritmo. Si llevamos muchas iteraciones sin mejorar, aumentamos el tamaño para forzar más exploración; si estamos mejorando, lo reducimos para explotar la zona prometedora.
- **Simulated Annealing**: en lugar de prohibir movimientos, acepta soluciones peores con cierta probabilidad que va bajando con el tiempo. Para problemas de scheduling como éste suele dar buenos resultados con una implementación más sencilla.
- **Algoritmos Genéticos**: trabajan con una población de soluciones y aplican cruce y mutación. Más costosos computacionalmente pero pueden explorar mejor espacios muy grandes.

Variaciones del problema

- **Temporada completa**: si en vez de una jornada consideramos las 38, aparecen restricciones nuevas de equidad (que ningún equipo juegue siempre en viernes o lunes) y de variedad en los horarios. La lista tabú podría adaptarse para registrar también combinaciones partido-horario a lo largo de toda la temporada.
- **Derechos televisivos**: en la realidad ciertos partidos tienen que emitirse en franjas concretas por contratos con operadores. Eso añade restricciones duras que habría que incorporar como criterios adicionales de validez.
- **Escalar a más equipos o ligas**: con más partidos por jornada el espacio crece exponencialmente y la fuerza bruta se vuelve todavía más inviable. Ahí es donde las metaheurísticas como Tabú realmente muestran su valor.

Validación del modelo

Un aspecto que no hemos abordado es comprobar si los coeficientes de audiencia del enunciado se ajustan a datos reales. Sería interesante contrastar con datos históricos de audiencia de LaLiga para ver si el modelo es realista o habría que recalibrar los parámetros.

```
Empieza a programar o a crear código con IA.
```